

Setlist

A web archive for DJ sets, where every mix carries its full tracklist and every track links out to where it can be heard.

MODELS	RELATIONS	PIVOT TABLE	SEEDING
5	$4 \times 1\text{-}N \cdot 1 \times N\text{-}M$	mix_track	Factory per model

01 Description

A DJ records a set and uploads it as an MP3 or WAV file. Visitors can then see which track plays at which minute, who made it, at what tempo, in which alphanumeric key, and where to find or buy it — everything a printed tracklist on a flyer could never hold.

From any track a visitor reaches the artist behind it and the other sets it appears in; the same track shows up in many mixes, which is what makes the archive worth browsing. Listening and reading need no account. Registered users can additionally comment on any set and upload sets of their own.

Because every track carries its BPM, its key in Camelot notation, its length and its genre, a tracklist here reads as a **mixing document** rather than a list of names: the tempo curve across a set becomes visible, and neighbouring keys show whether two tracks blend harmonically. The DJ supplies these values on upload — they already hold them in their own library. A track that is **already in the archive** is reused, not duplicated, so every new set tightens the network between the ones already there.

- Browse sets, tracks and artists through cross-linked detail pages
- Play a set while following its tracklist with start timestamps

- Read tempo, key, length and genre for every track in a set
-
- Follow any track to Spotify, SoundCloud or Bandcamp
-
- Registered users upload, edit and delete their own sets and build the tracklist
-
- Registered users comment on sets
-
- Admins manage the full catalogue and all user accounts

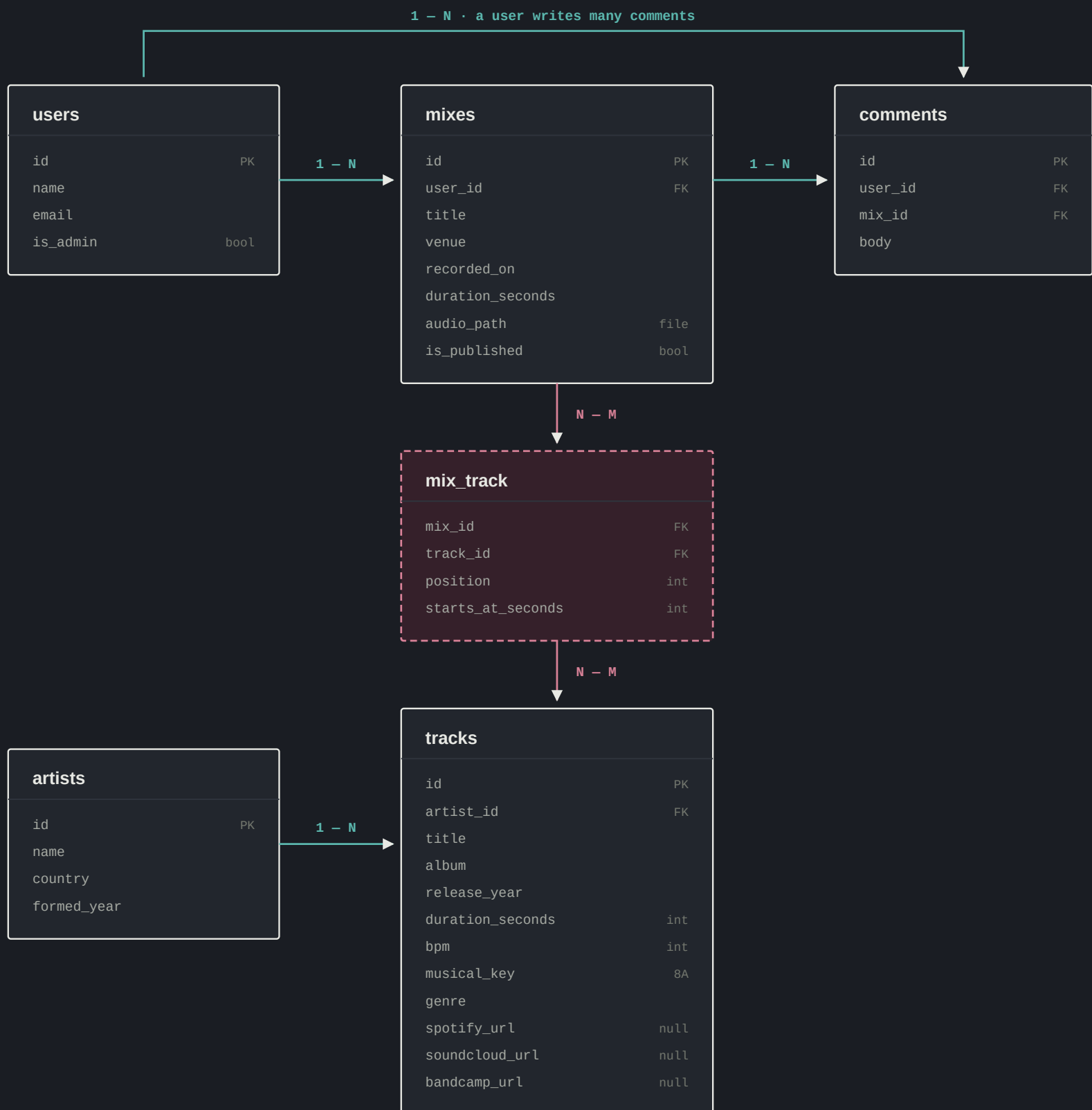
02 Who does what

ROLE	IN THE DATABASE	CAN DO
Visitor	Not stored — simply nobody logged in	Browse sets, tracks and artists. Play any published set and read its tracklist. Writes nothing.
Registered user	A row in <code>users</code>	Everything a visitor can, plus: comment on any set, and upload sets of their own.
DJ	Not a separate model. A registered user who has uploaded at least one mix	Upload, edit and delete their own sets and tracklists — only their own.
Admin	A row in <code>users</code> with <code>is_admin = true</code>	Manage every set, track, artist and account in the system.
Artist	A row in <code>artists</code> — no account, never logs in	Nothing. Credited as the maker of a track; exists purely as reference data.

DJ is not a role in the database. Every registered user may upload, and the moment they do, the app credits them as the DJ of that mix. No extra column is needed — the question "is this user a DJ?" is answered by `$user->mixes()->exists()`. Their account name is the name shown on their sets.

Artist is not a user. The musician credited on a track never signs in and has no account. Keeping the two apart is deliberate: the person who plays a track in a set and the person who produced it are rarely the same, and the archive has to name both.

03 Data structure



Solid teal: one row on the left belongs to many rows on the right Dashed red: the pivot table that carries the many-to-many relation, and two columns of its own – the position of a track in the mix and the second it starts at.

04

Models and their properties

User

A registered account. Uploads mixes and writes comments. Laravel's standard model, extended by one flag.

id	PK
name	string
email	string, unique
password	string, hashed
is_admin	boolean
timestamps	datetime

Mix

One recorded DJ set, uploaded as an audio file by the user who played it.

id	PK
user_id	FK → users
title	string
venue	string, nullable
recorded_on	date
duration_seconds	integer
audio_path	string
is_published	boolean
timestamps	datetime

Track

A single piece of music. Exists once and can appear in any number of mixes. Carries the values a DJ mixes by; the three URLs answer where a listener can get it.

id	PK
artist_id	FK → artists
title	string
album	string, nullable
release_year	integer, nullable
duration_seconds	integer
bpm	integer, nullable
musical_key	string(3), Camelot
genre	string, nullable
spotify_url	string, nullable
soundcloud_url	string, nullable
bandcamp_url	string, nullable
timestamps	datetime

Artist

The musician or group behind a track. Reference data, kept separate so tracks by the same artist can be found together.

id	PK
name	string
country	string, nullable
formed_year	integer, nullable
timestamps	datetime

Comment

A reaction to a mix. Only registered users can write one, so it always belongs to both a user and a mix.

id	PK
user_id	FK → users
mix_id	FK → mixes
body	text
timestamps	datetime

mix_track

Pivot table, not a model of its own. Carries the tracklist: which track sits at which position in which mix, and when it starts.

mix_id	FK → mixes
track_id	FK → tracks
position	integer
starts_at_seconds	integer

Naming of the relations

User — Mix

1 – N

→ A **User** hasMany mixes

→ A **Mix** belongsTo a user (the DJ who uploaded it)

User — Comment

1 – N

→ A **User** hasMany comments

→ A **Comment** belongsTo a user

Mix — Comment

1 – N

→ A **Mix** hasMany comments

→ A **Comment** belongsTo a mix

Artist — Track

1 – N

→ An **Artist** hasMany tracks

→ A **Track** belongsTo an artist

Mix — Track

N – M

→ A **Mix** belongsToMany tracks

→ A **Track** belongsToMany mixes

Resolved through the pivot table `mix_track`. The pivot carries two extra columns, `position` and `starts_at_seconds`, because the same track sits at a different place and a different minute in every mix. Read in Eloquent with `withPivot()`.

Pages (routes)

```
// public
welcome                                -> homepage, landing page

mixes.index                            -> list of all sets
mixes.show(id)                         -> one set: player, tracklist, comments
```

```
tracks.index          -> list of all tracks
tracks.show (id)      -> one track: where to listen, in which sets it appears
```

```
artists.index         -> list of artists
artists.show (id)     -> one artist including their tracks
```

```
about · contact
```

```
// authentication
login · register · logout
```

```
// user area (logged in, own content only)
user.dashboard        -> overview of my uploaded sets
```

```
GET    user.mixes.index    -> list of my sets
GET    user.mixes.create   -> empty form to upload a new set
POST   user.mixes.store    -> save the uploaded set
GET    user.mixes.edit (id) -> edit one of my sets, incl. its tracklist
PATCH user.mixes.update    -> save the input from the edit form
DELETE user.mixes.destroy   -> delete one of my sets
```

```
POST   mixes.comments.store -> post a comment on a set
```

```
// admin area (all content)
admin.dashboard
```

```
// Mix CRUD
GET    admin.mixes.index   -> list of all existing sets
GET    admin.mixes.create  -> empty form to create a set
POST   admin.mixes.store   -> save the input from the create form
GET    admin.mixes.edit (id) -> edit an existing set
PATCH admin.mixes.update   -> save the input from the edit form
DELETE admin.mixes.destroy  -> delete an existing set
```

```
// Track CRUD (same routes as mixes)
admin.tracks.xxxxx
```

```
// Artist CRUD (same routes as mixes)
admin.artists.xxxxx
```

```
// User CRUD (same routes as mixes)
admin.users.xxxxx
```

Against the expectations

EXPECTATION	IN THIS APP	
At least 3 models, including User	5 — User, Mix, Track, Artist, Comment	✓
Each model has at least one relationship	User 2 · Mix 3 · Track 2 · Artist 1 · Comment 2	✓
At least one one-to-many relationship	Four of them, listed in section 05	✓
Seedable with test data through factories	UserFactory · MixFactory · TrackFactory · ArtistFactory · CommentFactory, wired up in DatabaseSeeder, which also fills the mix_track pivot	✓